

```
File Edit Options Buffers Tools Help
hey my name is ankur Aggarwal
you buddy howz you
```

Figure 6: ispell-buffer screenshot



Note: Not all minor modes display an indicator on the mode-line; some minor modes are self-evident, such as the Tool Bar mode, which displays the graphical tool bar at the top of the Emacs frame. You can get a description of the mode by typing `Ctrl-h m`. You will get something like the following code:

To get rid of the help window, use `Ctrl-x 1`.

Important major modes

Fundamental mode (function name: *fundamental-mode*): Emacs's default mode with minimal settings and bindings, which we've already mentioned.

- Text mode (function name: *text-mode*): A basic mode for text editing.
- C mode (function name: *c-mode*): For editing C programs, a favourite of C programmers.
- Wordstar mode (function name: *wordstar-mode*): This special mode provides you the key-bindings of the old but popular WordStar editor.
- Paragraph-Indent Text (function name: *paragraph-indent-text-mode*): This mode is a special variation of the text mode, where the paragraph-movement commands work for paragraphs whose first lines are indented, and not just for paragraphs separated by blank lines.

Besides these, there are also TeX mode, for editing TeX docs; and lisp-interaction mode, for editing and compiling Lisp code. You can activate any major mode by typing `Alt+x` followed by the mode's function name.

Fun with minor modes

As mentioned earlier, minor modes are not restricted to a particular major mode, but can be applied to any major mode. Let's explore minor modes, with different scenarios to make it interesting.

Scenario 1: If you're a Java programmer, you have to be familiar with the line `System.out.println "text" that's used to print text to standard output. Tired of typing the same line over and over again?`

The solution is a minor mode named Abbrev (function name: *abbrev-mode*). After you invoke this mode, to assign the abbreviation "sop" for the string "`System.out.println`", go to the buffer and type "sop". Next, type `Alt+x inverse-add-global-abbrev`. Emacs will ask you to define the expansion for "sop" (see Figure 5). Type "`System.out.println`" and hit `Enter`. The next time you enter `sop` in the buffer, it will automatically

```
File Edit Options Buffers Tools Help
(0) how (1) Howe (2) howl (3) hows (4) how z (5)
-- *Choices* -- word: howz -- dict: american --
hey i am ankur aggarwal
you buddy
```

Figure 7: Flyspell-mode screenshot

change it to `System.out.println`. Finally, it will ask you whether or not to save the defined abbreviation for future sessions.

To do away with all the defined abbreviations, use the command `Alt+x kill-all-abbrevs`.

Scenario 2: You want spell-check in the editor

IsPELL, an interactive UNIX spelling checker, is built into Emacs, and is a powerful and convenient way to check buffers for misspelled words. To spell-check a word at the cursor, use the *ispell-word* function, or its key binding, `M- $\$$` . To spell-check a region, use the *ispell-region* function. To spell-check a whole buffer, use the *ispell-buffer* function.



Note: ispell gives you a word-by-word spell check, as seen in Figure 6. To highlight all the spelling mistakes in the buffer at one time, use the *flyspell-mode* function. The result is like what is shown in Figure 7.

Scenario 3: You want to automate the repetitive invocation of an Emacs command. Let's suppose you have selected a region that you want to move through 10 spaces. To move it a single space, I'd use `Ctrl-x Ctrl-i`—but I need to do that 10 times. Hard on the hands and patience!

Emacs provides a way to repeat a command *n* number of times; in this scenario, I use `Ctrl-u 10 Ctrl-x Ctrl-i`. This results in the move-region binding being invoked 10 times, doing my job. `Ctrl-u` is a 'universal' binding, and can be used with other key combinations to repeat it for a given number of times.

Scenario 4: You want to find a string in the buffer.

Use the various search functions/bindings:

isearch-forward will incrementally search the buffer forwards from the cursor.

isearch-backward will search backward from the cursor.

Ctrl-s or **Ctrl-r** will show all the matching words in the buffer (performs a non-incremental search).

Well that's it, for my basic introduction to Emacs.

I hope you liked it, and decide to give Emacs a try. Suggestions and questions are always welcome so feel free to contact me! 

By: Ankur Aggarwal

The author loves to explore open source technologies and enjoys programming. He has learned C, C++, core Java, and Python. He loves rock/metal music and has been trying his hand at the guitar. He has also started a blog recently: www.flossstuff.wordpress.com and can be contacted via coolankur2006@gmail.com or on Twitter as @ankurtwi.